Laporan Proyek Mini

Sistem Reservasi Tiket Bioskop Berbasis gRPC

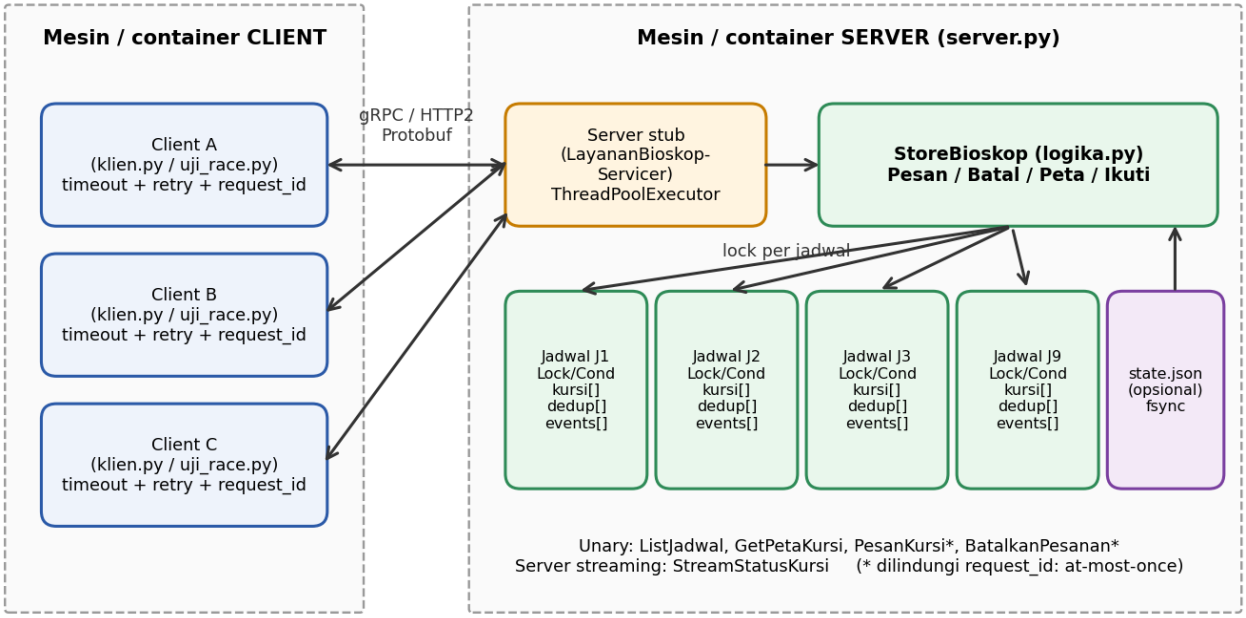
Mata Kuliah Komputasi Paralel dan Terdistribusi - Topik Remote Procedure Calls

Kelompok	[ISI nomor kelompok]
Anggota	[ISI nama - NIM] (2-3 orang)
Kelas	[ISI]
Studi kasus	No. 1 - Reservasi tiket bioskop (rebutan kursi yang sama)
Teknologi	Python 3.10+, gRPC, Protocol Buffers

Catatan: bagian bertanda [ISI] dilengkapi kelompok setelah menjalankan pengujian pada mesin masing-masing.

1. Deskripsi Studi Kasus dan Arsitektur

Sebuah bioskop memiliki beberapa jadwal tayang, masing-masing dengan denah kursi. Banyak pengguna (client) dapat memesan kursi pada saat yang sama sehingga terjadi **rebutan sumber daya bersama**: dua client tidak boleh sama-sama berhasil memesan kursi yang sama. Layanan juga harus tetap benar ketika jaringan lambat atau balasan hilang (client mengulang permintaan), dan pengguna dapat memantau perubahan status kursi secara real-time.



Gambar 1. Komponen dan alur pemanggilan RPC.

Alur satu pemesanan: (1) program client memanggil `pesan_kursi()` pada client stub; (2) stub melakukan marshalling `PesanRequest` (termasuk `request_id`) ke Protobuf; (3) dikirim lewat HTTP/2 dengan deadline; (4) server stub melakukan unmarshalling dan memanggil `StoreBioskop.pesan()`; (5) logika bisnis mengunci jadwal, memeriksa dan menulis kursi, mencatat hasil pada tabel deduplikasi; (6) `PesanResponse` dikemas dan dikirim kembali. Jika balasan hilang, client mengulang dengan `request_id` yang sama dan server menjawab dari tabel dedup.

2. Antarmuka Layanan (.proto)

Antarmuka mendefinisikan **5 RPC** (syarat minimal 4, termasuk minimal 1 streaming):

RPC	Jenis	Idempoten?	Penjelasan
ListJadwal	Unary	Ya	Daftar jadwal dan jumlah kursi kosong.
GetPetaKursi	Unary	Ya	Snapshot denah kursi satu jadwal.
PesanKursi	Unary	**Tidak**	Memesan 1-6 kursi, semua-atau-tidak-sama-sekali. Wajib request_id.
BatalkanPesanan	Unary	**Tidak**	Membatalkan pesanan (hanya pemilik). Wajib request_id.
StreamStatusKursi	Server streaming	Ya	Snapshot awal lalu setiap perubahan kursi secara real-time.

Penanganan galat memakai kode status gRPC: `NOT_FOUND` (jadwal/kode tidak ada), `INVALID_ARGUMENT` (input tidak valid, request_id dipakai ulang untuk isi berbeda). Kegagalan bisnis yang deterministik (kursi sudah terisi) dikembalikan sebagai `sukses=false` agar ikut tersimpan dalam cache deduplikasi.

Isi lengkap `bioskop.proto`:

```
syntax = "proto3";

package bioskop;

// Kontrak layanan reservasi tiket bioskop.
service LayananBioskop {
  // Unary, idempoten: daftar jadwal tayang beserta jumlah kursi kosong.
  rpc ListJadwal (ListJadwalRequest) returns (ListJadwalResponse);

  // Unary, idempoten: peta kursi sebuah jadwal (snapshot saat ini).
  rpc GetPetaKursi (PetaRequest) returns (PetaKursi);

  // Unary, TIDAK idempoten: memesan satu atau beberapa kursi (semua-atau-tidak-sama-sekali).
  // Dilindungi request_id -> semantik at-most-once.
  rpc PesanKursi (PesanRequest) returns (PesanResponse);

  // Unary, TIDAK idempoten: membatalkan pesanan. Dilindungi request_id.
  rpc BatalkanPesanan (BatalRequest) returns (BatalResponse);

  // Server streaming: snapshot awal lalu setiap perubahan status kursi secara real-time.
  rpc StreamStatusKursi (PetaRequest) returns (stream PerubahanKursi);
}

enum StatusKursi {
  KOSONG = 0;
  TERISI = 1;
}

message ListJadwalRequest {}

message Jadwal {
  string id = 1;
  string film = 2;
  string studio = 3;
  string waktu = 4;
  int32 baris = 5;
  int32 kolom = 6;
  int32 kursi_kosong = 7;
}

message ListJadwalResponse { repeated Jadwal jadwal = 1; }

message PetaRequest { string jadwal_id = 1; }

message Kursi {
  string kode = 1;            // contoh: "A1"
  StatusKursi status = 2;
  string pemesan = 3;         // kosong jika KOSONG
}

message PetaKursi {
  Jadwal jadwal = 1;
  repeated Kursi kursi = 2;
}

message PesanRequest {
  string request_id = 1;      // UUID unik per operasi logis (sama untuk setiap retry)
  string jadwal_id = 2;
  string pemesan = 3;
  repeated string kursi = 4;
}

message PesanResponse {
  bool sukses = 1;
  string pesan = 2;
  string kode_booking = 3;
  repeated string kursi_dipesan = 4;
  repeated string kursi_gagal = 5;
  bool duplikat = 6;          // true jika balasan berasal dari cache deduplikasi
}

message BatalRequest {
  string request_id = 1;
  string kode_booking = 2;
  string pemesan = 3;
}

message BatalResponse {
```

```
  bool sukses = 1;
  string pesan = 2;
  repeated string kursi_dilepas = 3;
  bool duplikat = 4;
}

message PerubahanKursi {
  int64 seq = 1;              // nomor urut perubahan (0 untuk snapshot awal)
  Kursi kursi = 2;
  bool snapshot = 3;          // true = bagian dari snapshot awal
}
```

3. Strategi Concurrency dan Toleransi Kegagalan

3.1 Concurrency

Server gRPC memproses permintaan pada `ThreadPoolExecutor`, sehingga banyak permintaan mengakses data kursi bersamaan. Pilihan mekanisme:

Pilihan	Alasan
Lock per jadwal (threading.Condition)	Pemesanan pada jadwal berbeda berjalan paralel; pada jadwal yang sama diserialkan. Lebih baik daripada lock global (bottleneck) dan lebih sederhana serta bebas deadlock dibanding lock per kursi.
Cek + tulis + catat hasil dalam satu critical section	Menutup celah check-then-act: tanpa ini dua thread bisa sama-sama melihat kursi kosong.
Semua-atau-tidak-sama-sekali	Pemesanan banyak kursi tidak pernah sebagian terisi.
Condition yang sama untuk streaming	Pemantau menunggu dengan wait(); setiap perubahan memanggil notify_all(), tanpa polling.

3.2 Toleransi kegagalan

Gangguan	Penanganan
Balasan lambat / hilang	Client memakai deadline (timeout) per panggilan.
Server sementara tidak tersedia	Retry hanya untuk DEADLINE_EXCEEDED dan UNAVAILABLE dengan exponential backoff (0,25 x 2^(n-1) s, maks 4 s) dan jitter; maksimal 4 percobaan.
Retry menyebabkan eksekusi ganda	request_id (UUID) dibuat sekali per operasi logis; server menyimpan hasil per request_id sehingga permintaan ulang mengembalikan hasil lama (**at-most-once**). request_id yang dipakai ulang untuk isi berbeda ditolak.
Server restart	Opsi --state-file: state dan tabel dedup ditulis atomik (os.replace + fsync) sebelum balasan dikirim, sehingga jaminan at-most-once bertahan.
Client terputus saat streaming	Server memeriksa context.is_active() dan menghentikan stream; client dapat menyambung ulang dan menerima snapshot baru.

Operasi idempoten (ListJadwal, GetPetaKursi) aman diulang tanpa pengaman tambahan; operasi tidak idempoten (PesanKursi, BatalkanPesanan) wajib memakai request_id sesuai ketentuan tugas.

4. Hasil Pengujian dan Analisis

4.1 Uji unit logika bisnis (tanpa jaringan)

`python -m unittest -v test_logika` - **11 uji, semuanya lulus**:

Uji	Yang dibuktikan
race_satu_kursi_hanya_satu_pemenang	200 thread serentak merebut A1 -> tepat 1 pemenang.
beban_acak_tidak_ada_kursi_ganda	150 thread masing-masing meminta 3 kursi acak -> tidak ada kursi ganda, jumlah terisi konsisten.
tanpa_lock_terjadi_pemesanan_ganda	Kontrol negatif: tanpa lock, lebih dari satu pemenang.
dedup_request_id_sama / dedup_retry_serentak	Permintaan ulang (juga serentak, 30 thread) hanya dieksekusi sekali.
request_id_dipakai_untuk_permintaan_lain	request_id yang sama untuk isi berbeda ditolak.
semua_atau_tidak_sama_sekali	Pesanan A1+A2 saat A1 terisi -> A2 tetap kosong.
batal	Hanya pemilik; batal ganda ditolak; retry batal aman.
stream_snapshot_dan_perubahan	Snapshot 80 kursi lalu perubahan diterima berurutan.
persistensi_dedup_setelah_restart	Setelah 'restart', request_id lama tetap dikenali.
input_tidak_valid	Validasi request_id, kode kursi, jadwal.

4.2 Eksperimen lock vs tanpa lock

`python eksperimen_lokal.py`: 100 thread merebut kursi A1, 10 ronde (hasil percobaan nyata):

Konfigurasi	Pemenang per ronde	Ronde dengan pemesanan ganda
Dengan lock	1, 1, 1, 1, 1, 1, 1, 1, 1, 1	0 / 10
Tanpa lock	97, 100, 100, 100, 100, 100, 100, 100, 100, 100	10 / 10

Analisis: tanpa lock, interval antara pemeriksaan 'kursi kosong' dan penulisan pemilik adalah celah race condition: hampir semua thread melihat kursi kosong sebelum ada yang menulis, sehingga satu kursi 'terjual' ke puluhan orang. Dengan lock, pemeriksaan dan penulisan atomik sehingga tepat satu thread menang. (Angka tanpa lock sengaja diperbesar dengan jeda 1 ms di dalam celah agar mudah diamati; pada beban nyata celahnya lebih sempit tetapi tetap ada.)

4.3 Uji race condition melalui gRPC (dua mesin)

Jalankan `python uji_race.py --host IP:50051 --klien 50` dan tempel keluarannya:

Tes	Hasil yang diharapkan	Hasil kelompok
Tes 1: 50 client rebut J1/A1	pemenang = 1	[ISI]
Tes 2: 100 client x 3 kursi acak (J2)	kursi ganda = tidak ada; jumlah konsisten	[ISI]
Pembanding: server --tanpa-lock	pemenang > 1 / kursi ganda ada	[ISI]

4.4 Demo timeout, retry, dan deduplikasi

Skenario (server --lambat-prob 0.5)	Retry	Sukses (client)	Terisi (server)	Konsisten?
demo_retry.py (id tetap)	[ISI]	[ISI]	[ISI]	[ISI] (harapan: ya)
demo_retry.py --tanpa-dedup	[ISI]	[ISI]	[ISI]	[ISI] (harapan: tidak)

4.5 Kinerja: latensi dan throughput

`python bench.py --host IP:50051 --n 100` menghasilkan `hasil/hasil_bench.csv` dan grafik `hasil/hasil_bench.png` (sisipkan di bawah). Mode baca = GetPetaKursi; mode tulis = PesanKursi + BatalkanPesanan.

Mode	Client	Total op	Waktu (s)	Throughput (req/s)	Latensi rata-rata (ms)	p95 (ms)
baca	1	[ISI]	[ISI]	[ISI]	[ISI]	[ISI]
baca	5	[ISI]	[ISI]	[ISI]	[ISI]	[ISI]
baca	20	[ISI]	[ISI]	[ISI]	[ISI]	[ISI]
tulis	1	[ISI]	[ISI]	[ISI]	[ISI]	[ISI]
tulis	5	[ISI]	[ISI]	[ISI]	[ISI]	[ISI]
tulis	20	[ISI]	[ISI]	[ISI]	[ISI]	[ISI]

Panduan analisis (isi dengan data sendiri): bandingkan throughput 1 vs 5 vs 20 client; jelaskan mengapa throughput tidak naik linear (GIL Python membatasi paralelisme CPU; lock per jadwal menyerialkan penulisan pada jadwal yang sama; jumlah thread server `--workers`; latensi jaringan). Bandingkan latensi baca (tanpa kontensi) dengan tulis (ada kontensi lock). Ulangi dengan `--workers 1/4/32` untuk melihat pengaruh thread.

[ISI: sisipkan grafik hasil_bench.png dan analisis penyebab]

5. Pembagian Tugas Anggota

Anggota	Tugas	Commit utama
[ISI]	[ISI contoh: desain .proto, server, logika concurrency]	[ISI]
[ISI]	[ISI contoh: client, retry/dedup, demo]	[ISI]
[ISI]	[ISI contoh: pengujian kinerja, laporan, video]	[ISI]

6. Tautan dan Referensi

Item	Tautan / keterangan
Repositori Git	[ISI URL repositori; riwayat commit setiap anggota]
Video demo (maks. 5 menit)	[ISI URL video]
Cara menjalankan	Lihat README.md (instalasi, generate stub, menjalankan server/client, Docker)
Alat bantu	Kerangka kode dan dokumen dibuat dengan bantuan asisten AI Claude (Anthropic); kelompok telah menelaah, menjalankan, dan menyesuaikan kode. Referensi: dokumentasi gRPC Python (grpc.io) dan modul praktikum RPC.